ANKARA YILDIRIM BEYAZIT UNIVERSITY
GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES



THESIS ss

M.Sc. Thesis by

Muhammed Yasinhan YAŞAR

Department of Computer Engineering

September, 2023

ANKARA

THESIS ss

A Thesis Submitted to the

Graduate School of Natural And Applied Sciences of

Ankara Yildirim Beyazit University

In Partial Fulfillment of the Requirements for the Degree of Master of Science in Computer Engineering, Department of Computer Engineering

by

Muhammed Yasinhan YAŞAR

September, 2023

ANKARA

M.Sc. THESIS EXAMINATION RESULT FORM

We have read the thesis entitled "**THESIS ss**" completed by **MUHAMMED YASINHAN YAŞAR** under supervision of **Assoc. Prof. MUHAMMED ABDULLAH BÜLBÜL** and we certify that in our opinion it is fully adequate, in scope and in quality, as a thesis for the degree of Master of Science.

Assoc. Prof. Muhammed Abdullah BÜLBÜL

Supervisor

Prof. Dr. Cccccc CCCCC

Jury Member

Prof. Dr. Ddddd DDDDD

Jury Member

Prof. Dr. Aaaa AAAA

Director

Graduate School of Natural and Applied Sciences

ETHICAL DECLARATION

I hereby declare that, in this thesis which has been prepared in accordance with the Thesis Writing Manual of Graduate School of Natural and Applied Sciences,

- All data, information and documents are obtained in the framework of academic and ethical rules,
- All information, documents and assessments are presented in accordance with scientific ethics and morals,
- All the materials that have been utilized are fully cited and referenced,
- No change has been made on the utilized materials,
- All the works presented are original,

and in any contrary case of above statements, I accept to renounce all my legal rights.

Date: 2023, 20 September **Signature:** _____

Name & Surname: Muhammed Yasinhan YAŞAR

ACKNOWLEDGEMENTS

Firstly, I would like to express my sincere gratitude to my advisor Prof. Asds ASDAS for continuous support of...

Beside my advisor, I am deeply indebted to Prof. Asdas

I thank my colleagues at AYBU...

September, 2023 **Muhammed Yasinhan YAŞAR**

THESIS ss

ABSTRACT

Imaging in sub-Terahertz (THz) band has become an attractive research area withemerging technology since it has a wide range of potential applications in defense,security or airborne systems...

Keywords: Compressed sensing, sub-THz, single pixel imaging, super-resolution, computational ghost imaging, millimeter wave, image quality assessment, convex optimization, matching pursuit, hadamard.

TÜRKÇE TEZ BAŞLIĞITÜRKÇE TEZ BAŞLIĞITÜRKÇE TEZ BAŞLIĞITÜRKÇE TEZ BAŞLIĞITÜRKÇE TEZ BAŞLIĞI

ÖZ

Terahertz (THz)-altı bandında görüntüleme, savunma, güvenlik veya havacılık sistemlerinde çok çeşitli potansiyel uygulamalara sahip olması nedeniyle, gelişen teknolojiyle birlikte çekici bir araştırma alanı haline gelmiştir...

Anahtar kelimeler: Sıkıştırılmış algılama, THz-altı, tek piksel görüntüleme, süper-çözünürlük, hesaplamalı hayalet görüntüleme, milimetre dalga, görüntü kalitesi değerlendirmesi, konveks optimizasyon, eşleştirme algoritması, hadamard.

CONTENTS

M.Sc. THESIS EXAMINATION RESULT FORM ii
ETHICAL DECLARATION iii
ACKNOWLEDGEMENTS iv
ABSTRACT v
ÖZ vi
NOMENCLATURE viii
CONTENTS viii
LIST OF TABLES ix
LIST OF FIGURES x

CHAPTER 1 – INTRODUCTION 1

1.1 Section Title 1
1.1.1 Subsection title 1

CHAPTER 2 – BACKGROUND 2

2.1 Cellular Automata 2
2.1.1 Definition and Basic Principles 2
2.1.2 From Simple Rules to Complex Systems: Intuition and Examples 4
2.1.3 Cellular Automata as Convolutions 8
2.2 Neural Cellular Automata 10
2.2.1 Definition and Basic Principles 10

REFERENCES 11

APPENDICES 13

Appendix A – More GoL Units 14
Appendix B – GoL Logic Gates 15
Appendix C – Convolution Definiton 16

CURRICULUM VITAE 17

NOMENCLATURE

Roman Letter Symbols

F	Global update rule for CA state
f	Local update rule for cell state
G	d-dimensional Grid
N_{x_t}	Neighbor cell states of the current cell
S_t	Grid state
x_t	Current cell state

Acronyms

CA	Cellular Automata
GoL	Game of Life
LWSS	Lightweight Spaceship
NCA	Neural Cellular Automata

LIST OF TABLES

Table 2.1 Rule 0 configuration. 4
Table 2.2 Rule 18 configuration. 5
Table 2.3 Rule 30 configuration. 6
Table A.1 Örnek tablo....14
Table B.1 Örnek tablo....15

LIST OF FIGURES

Figure 1.1 SDF GNCA.....1
Figure 2.1 Rule 18 with different initial conditions.....5
Figure 2.2 Rule 30 with different initial conditions.....5
Figure 2.3 GoL Rule Table.....6
Figure 2.4 GoL Block and Beehive.....7
Figure 2.5 GoL Glider and LWSS 8
Figure 2.6 Representing Game of Life as convolution.....9
Figure 2.7 Representing Game of Life as multichannel convolutionan perceptron..9

CHAPTER 1

INTRODUCTION

Bu döküman Ankara Yıldırım Beyzıt Üniversitesi Fen Bilimleri Enstitüsünün Haziran 2021 kurallarına göre güncellenmiştir.

1.1 Section Title

This is a text inside the section.

1.1.1 Subsection title

After subsection text.

Example Figure

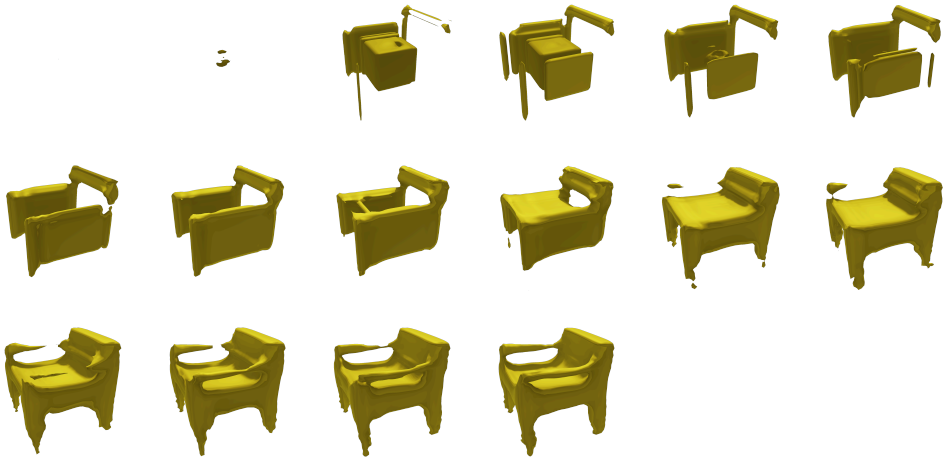


Figure 1.1 Emerging of an SDF chair model sampled from a learned distribution p_{data}

CHAPTER 2

BACKGROUND

This chapter outlines the foundational prerequisites. While formal proofs are omitted to maintain focus on practical and conceptual understanding, appropriate citations are provided for readers seeking rigorous mathematical treatments.

2.1 Cellular Automata

2.1.1 Definition and Basic Principles

Cellular Automata (CA) models were first introduced as self-replicating systems [1] inspired by biological systems. CA are d-dimensional spatio-temporal, locally interacting computational models that obey a handcrafted update rule.

Classical CA models are discrete, deterministic, and synchronous systems that typically operate on a uniform grid of cells $G \subseteq \mathbb{Z}^d$. Here, S_t represents the global state of the grid at time t, and F represents the deterministic update rule, which depends only on the previous state of the grid as formulated in Equation **2.1**.

$$S_{t+1} = F(S_t) \tag{**2.1**}$$

This formulation can be extended by introducing a history of past states [2, 3]. Given a memory capacity T, the update rule F can be redesigned to consider the previous $T+1$ states, as shown in Equation **2.2**.

$$S_{t+1} = F(S_t, S_{t-1}, \ldots, S_{t-T}) \tag{**2.2**}$$

This establishes the behavior of the system from a global perspective. To analyze the local and emergent behavior of a Cellular Automaton, it is necessary to examine the underlying dynamics from the perspective of a single cell.

Let $x_t \in S_t$ be the state of a specific cell in the grid $G \subseteq \mathbb{Z}^d$, and let $N_{x_t} = [x_1, x_2, \ldots, x_n]$ be the sequence of states of its neighboring cells, where n is the size of the defined neighborhood. The updated state of this individual cell is given in Equation **2.3**.

$$x_{t+1} = f(x_t, N_{x_t}) \tag{2.3}$$

Therefore, the global grid state update rule F can be defined as the collective application of the local rule f to all cells, as expressed in Equation **2.4**.

$$F(S_t) = \{f(x_t, N_{x_t}) \mid \forall x_t \in S_t\} \tag{2.4}$$

Furthermore, this framework is not limited to scalar representations [4]. By introducing multiple channels the spatial grid expands into a multi-dimensional feature space. Specifically, while the spatial domain remains a subset of $\mathbb{Z}^d$, the global state of the grid becomes a vector field mapping $S_t : \mathbb{Z}^d \to \mathbb{R}^C$.

The cell state is naturally extended to encompass these multiple channels, defined as a vector $\mathbf{x}_t \in \mathbb{R}^C$. Consequently, the neighborhood of a cell is no longer a sequence of scalars, but a sequence of C-dimensional vectors. Let $\mathbf{N}_{\mathbf{x}_t} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n]$ represent this multi-channel neighborhood. The local update rule f is thus redefined to map the current vector state and its vector neighborhood to a new vector state, as shown in Equation **2.5**.

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{N}_{\mathbf{x}_t}) \tag{2.5}$$

Similarly, let $\mathbf{S}_t$ represent the global state of the grid where every cell contains a C-dimensional vector. The global update rule F is redefined as the set of all local multi-channel updates applied across the entire grid, expressed in Equation **2.6**.

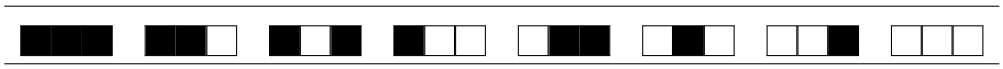
$$\mathbf{S}_{t+1} = F(\mathbf{S}_t) = \{f(\mathbf{x}_t, \mathbf{N}_{\mathbf{x}_t}) \mid \forall \mathbf{x}_t \in \mathbf{S}_t\} \tag{2.6}$$

For a deeper understanding of CA's historical advancements from a perspective of reversibility, conservation laws, decidability questions, computational universality and limit behavior see [5].

2.1.2 From Simple Rules to Complex Systems: Intuition and Examples

Wolfram initiated the first systematic computational and theoretical study of one-dimensional ceullular automata [6] and argued the local and global properties of them.

Chosen $d = 1$ for grid $G \subseteq \mathbb{Z}^d$ is a one-dimensional celullar automata, $f(x_t, N_{x_t})$ is shown as a rule table in Table 2.1.



111	110	101	100	011	010	001	000
0	0	0	0	0	0	0	0

Table 2.1 Rule 0 configuration.

For state **101** $x_t = 0,\ N_{x_t} = [1,1],\ n = 2$

For state **011** $x_t = 1,\ N_{x_t} = [0,1],\ n = 2$

For state **010** $x_t = 1,\ N_{x_t} = [0,0],\ n = 2$

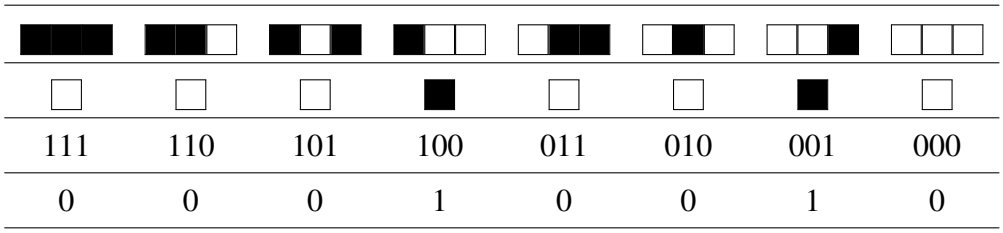
…

And since all the next states $x_{t+1} = 0$ update rule for Table 2.1 which means all the values at the last row are 0, update rule $f(x_t, N_{x_t}) = 0$. And this is why it is called zero rule. Zero rule generates nothing for all possible combinations of initial states as expected.

But let's look at another rule slightly more interesting to look at it create patterns. Let's define update rule f as the following Equation **2.7**.

$$\begin{aligned} x_t = x_0,\ N_{x_t} &= [x_1, x_2] \\ f(x_0, [x_1, x_2]) &= x_0(x_1 + x_2 - 2x_1x_2) \end{aligned} \tag{2.7}$$

This definition of f generates the Rule 18 in Table 2.2



111	110	101	100	011	010	001	000
0	0	0	1	0	0	1	0

Table 2.2 Rule 18 configuration.

It gets this name because the last row of Table 2.2 represents number 18 in binary. And this simple rule is the generator of the following patterns under different initial states shown in Figure 2.1.

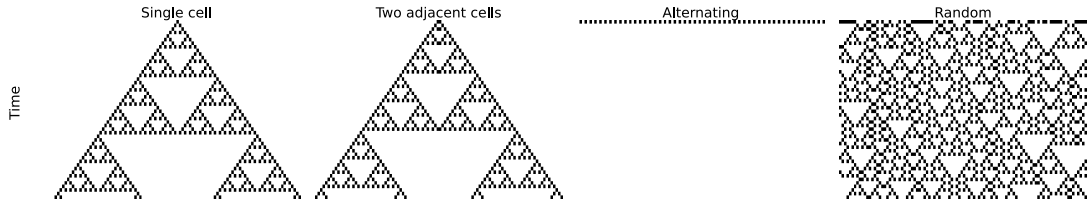


Figure 2.1 Rule 18 with different initial conditions.

We can redefine f to achieve even more chaotic structures. Following definition of f in Equation **2.8** achieves the famous Rule 30

$$f(x_0, [x_1, x_2]) = x_1 + x_0(x_2 - 2x_1x_2) \tag{2.8}$$

which yields the results in Figure 2.2 with the corresponding rule table Table 2.3

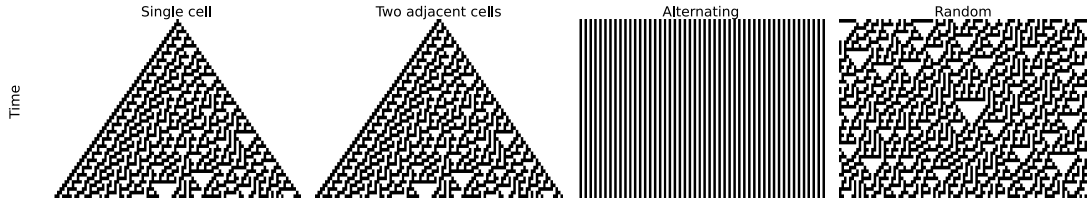


Figure 2.2 Rule 30 with different initial conditions.

Even with the initial condition of starting with a single cell, it is capable of creating patterns where the only way to predict a certain state of a cell x_t in timestep t is to run the system at least t times. Wolfram termed this property *computational irreducibility* [7]. This tells us Rule 30 produces patterns for which no shortcut is known rather than running the simulation.

111	110	101	100	011	010	001	000
0	0	0	1	1	1	1	0

Table 2.3 Rule 30 configuration.

As observed in the study the number of possible configurations the system can actually produce $2^{(2^3)} = 2^8 = 256$ for one-dimensional binary CA. However, a particular CA rule does not necessarily produce all of these configurations during its evolution. The complexity of CA can therefore be studied by examining the set of configurations that can occur and how this set grows with the system size. Wolfram [8] investigated this perspective in detail, introducing entropy and dimension-based measures for characterizing the complexity of CA.

This conceptual framework naturally extends to higher-dimensional spaces. Conway's Game of Life [9] is one of the most popular two-dimensional Cellular Automaton.

Let $G \subseteq \mathbb{Z}^d$, $d = 2$ and $N_{x_t} = [x_1, x_2, \ldots, x_n]$, $n = 8$

Game of Life (GoL) is a two-dimensional binary Cellular Automaton that uses Moore neighborhood (i.e., the eight cells surrounding a cell, including diagonals).

We can define the Game of Life rule table in the following Figure 2.3. Since the rule depends only on the number of alive neighbours showing all other reflections and configurations would be trivial.

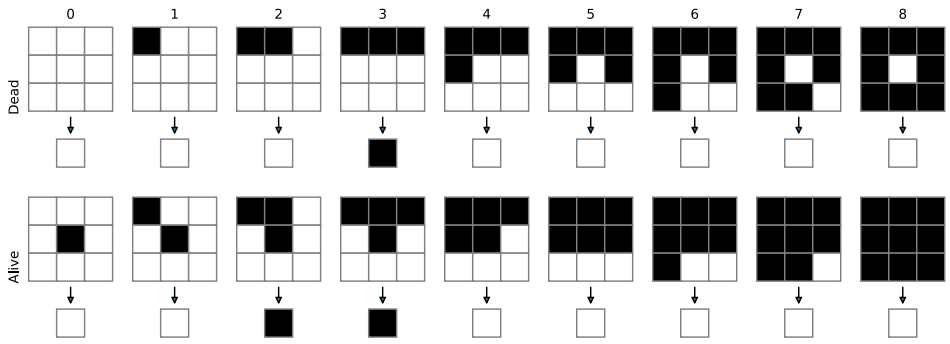
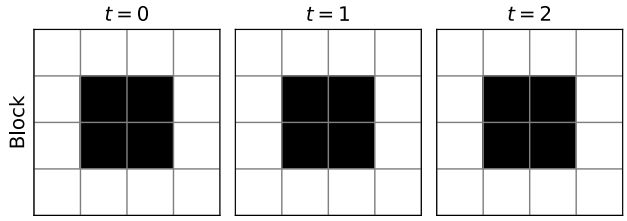


Figure 2.3 Game of Life Rule Table

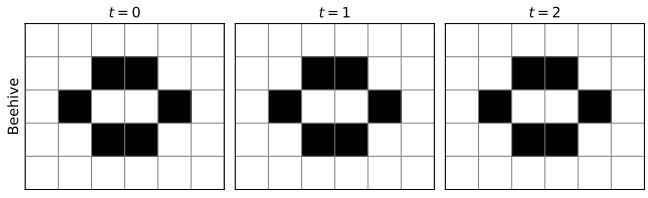
But it is a more compact and also more widespread representation to define the rule function f as the following partial function shown in Equation **2.9**.

$$
\begin{aligned}
n_{\text{alive}} &= \sum_{i=1}^{8} x_i \quad \text{(number of alive neighbors)} \\
f(x_0, [x_1, x_2, \ldots, x_8]) &= \begin{cases} 1 & x_0 = 1 \wedge (n_{\text{alive}} = 2 \vee n_{\text{alive}} = 3) \\ 1 & x_0 = 0 \wedge n_{\text{alive}} = 3 \\ 0 & \text{otherwise} \end{cases}
\end{aligned} \tag{2.9}
$$

Unlike the elementary one-dimensional CA, where all $2^{(2^3)} = 256$ rules can be analysed, the space of two-dimensional rules is much larger ($2^{(2^9)}$ for the Moore neighborhood), and Game of Life is only one particular rule in it. What makes it special is that Conway's Game of Life is proven to be computationally universal [10, 11]. To put it simply, a Turing machine can be simulated using the local construct units introduced by the Game of Life. These units are categorized in different ways based on their behavior.



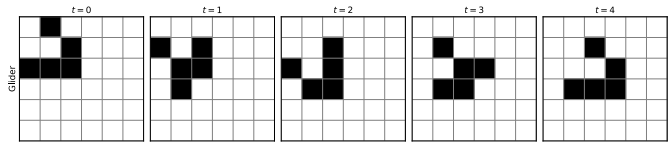
(a) Block unit, the most basic static unit.



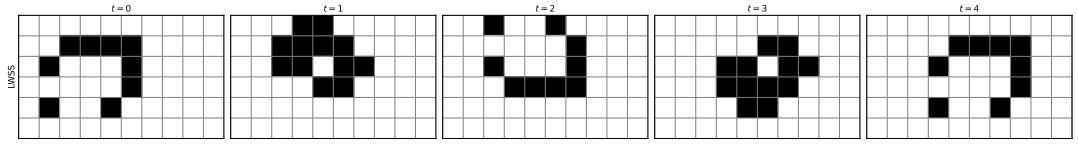
(b) Beehive unit, also static.

Figure 2.4 Unless there is external interference, these units do not change.

The units shown in Figure 2.4 are able to remain static by feeding their neighbors by staying alive. When we examine the block in Figure 2.4a we see that all individual cell has exactly 3 neighbors which keep it alive according to the first case in update rule Equation **2.9**, and Figure 2.4b also satisfy the same case by having $n_{alive} = 2 \vee n_{alive} = 3$.



(a) Glider unit, a diagonally moving unit that generally represents a bit.



(b) LWSS (lightweight spaceship) unit, a horizontal moving unit.

Figure 2.5 Moving units.

Figure 2.5 shows two GoL patterns, a glider and a LWSS, that exhibit movement-like behavior: the live cells are repeatedly re-created one position further along, so the pattern appears to travel across the grid even though no cell actually moves. Such moving patterns can carry signals, and collisions between streams of gliders implement logic gates [12,13]. Since NAND is functionally complete, GoL can in principle realize any boolean circuit, and indeed it is Turing complete [10,12]. For more figures of units and implementation of logic gates see Appendix A and B.

And this particular configuration of two-dimensional binary Cellular Automaton—Conway's Game of Life—shows us how extremely complex and chaotic systems can emerge from simple local rules.

2.1.3 Cellular Automata as Convolutions

Let's try to express GoL using convolution together with a thresholding function. Since each cell is affected only by its eight neighboring cells, a 3×3 convolution kernel will be sufficient. But we need to choose our kernel wisely to distinguish the cell and its neighbors. So the following Equation **2.10** shows a naive way of choosing a kernel.

Let the grid be $G \subseteq \mathbb{Z}^2$ and let $S_t : G \to \{0,1\}$ denote the state of the grid at time t.

$$k = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 9 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \qquad X_t = k * S_t \tag{2.10}$$

The convolution sums the cell itself, weighted by 9, and its eight neighbors weighted by 1 to be able to create the distinction of alive and dead cells. A threshold activation σ then gives the next state as following,

$$S_{t+1} = \sigma(k * S_t), \qquad \sigma(x) = \begin{cases} 1 & x \in \{3, 11, 12\}, \\ 0 & \text{otherwise.} \end{cases} \tag{2.11}$$

where σ is applied elementwise to the grid $X_t = k * S_t$ and returns 1 for every cell whose sum is 3, 11 or 12, and 0 for every other cell. The pipeline is expressed in Figure 2.6.

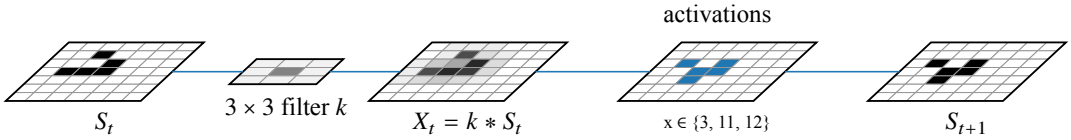


Figure 2.6 Representing Game of Life as convolution.

William Gilpin showed that any CA may readily be represented using a convolutional neural network with a network-in-network architecture [14]. Their idea extended our example naive pipeline in Figure 2.6 to a multi channel convolutional perceptron, that also learn the weights of the kernel—which means it learns the update rule.

Figure 2.7 and Equation **2.12** are hypotetical assumptions for intuitive understanding inspired by [14], the following inferences can be made:

The pipeline above relies on a hand-designed kernel and a hard threshold σ, which is not differentiable, so none of its weights can be learned by gradient descent. To make the rule learnable, replace the single kernel with several channels and a smooth activation.

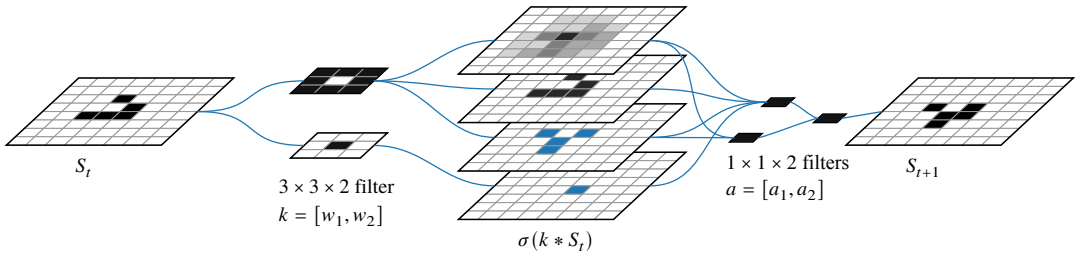


Figure 2.7 Representing Game of Life as multichannel convolutional perceptron.

Each channel has its own 3×3 kernel w_i that looks at the same neighborhood, and a 1×1 convolution with weights a_i then mixes the activated channels into the next state. This way no single kernel has to encode the whole rule (such as the hand-picked weight 9 for the center cell), and the network can learn the update rule from data. For two channels this gives

$$\begin{aligned} \theta &= [w_1, w_2, a_1, a_2] \\ S_{t+1} &= F_\theta(S_t) = a_1 \sigma(w_1 * S_t) + a_2 \sigma(w_2 * S_t) \end{aligned} \tag{2.12}$$

where θ is learnable parameters, and F is the global update function.

This idea was first introduced in [15]

2.2 Neural Cellular Automata

2.2.1 Definition and Basic Principles

REFERENCES

[1] Von Neumann J., Burks A. W., and Burks A. W., "Theory of self-reproducing automata," 1966.

[2] Letourneau P.-J., "Elementary cellular automata with memory," *Department of Physics, University of Calgary*.

[3] and Alonso-Sanz R. and Martín M. a., "Elementary cellular automata with memory," *Complex Systems*, vol. 14, no. 2, p. 99–126, 2003. [Online]. Available: http://dx.doi.org/10.25088/ComplexSystems.14.2.99

[4] D'humières D. and Lallemand P., "Lattice gas automata for fluid mechanics," *Physica A: Statistical Mechanics and its Applications*, vol. 140, no. 1-2, pp. 326–335, 1986.

[5] Kari J., "Theory of cellular automata: A survey," *Theoretical computer science*, vol. 334, no. 1-3, pp. 3–33, 2005.

[6] Wolfram S., "Statistical mechanics of cellular automata," *Reviews of modern physics*, vol. 55, no. 3, p. 601, 1983.

[7] ——, "Undecidability and intractability in theoretical physics," *Physical Review Letters*, vol. 54, no. 8, p. 735, 1985.

[8] ——, "Cellular automata as models of complexity," *Nature*, vol. 311, no. 5985, pp. 419–424, 1984.

[9] Conway J., "Conway's game of life," *Scientific American*, vol. 223, pp. 120–123, 1970.

[10] Rendell P., "Turing universality of the game of life," in *Collision-based computing*. Springer, 2002, pp. 513–539.

[11] ——, "Turing machine universality of the game of life," Ph.D. dissertation. [Online]. Available: https://uwe-repository.worktribe.com/output/822575

[12] Berlekamp E. R., Conway J. H., and Guy R. K., *Winning Ways for Your Mathematical Plays*, 2nd ed. AK Peters/CRC Press, 2004, vol. 4.

[13] Myreen M. O. and Carneiro M., "Gol in gol in hol: Verified circuits in conway's game of life," *arXiv preprint arXiv:2504.00263*, 2025.

[14] Gilpin W., "Cellular automata as convolutional neural networks," *Physical Review E*, vol. 100, no. 3, p. 032402, 2019.

[15] Wulff N. and Hertz J. A., "Learning cellular automaton dynamics with neural networks," *Advances in Neural Information Processing Systems*, vol. 5, 1992.

APPENDICES

Appendix A: More GoL Units

Appendix B: GoL Logic Gates

Appendix C: Convolution Definiton

Appendix A – More GoL Units

This is a text inside the section

Table A.1 Örnek tablo

A	1
B	2
C	3

Appendix B – GoL Logic Gates

This is a text inside the section

Table B.1 Örnek tablo

A	1
B	2
C	3

Appendix C – Convolution Definiton

$$k = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 9 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \qquad X_t = k * S_t \tag{0.13}$$

$$X_t(i, j) = \sum_{u=-1}^{1} \sum_{v=-1}^{1} k(u, v)\, S_t(i-u,\ j-v)$$

CURRICULUM VITAE

PERSONAL INFORMATION

Name Surname : Muhammed Yasinhan YAŞAR

Date of Birth : 01.01.1990

Phone :

E-mail :

EDUCATION

High School :

Bachelor :

Master Degree :

WORK EXPERIENCE

Research Assist. or Engineer etc. : T.C. bla bla

TOPICS OF INTEREST

- Thermodynamics
- Fluid Mechanics
- Heat Transfer
- Internal Combustion Engines

PRESENT ORGANIZED SOURCES / PRESENTATION

bla bla

September, 2023
ANKARA

Department of Computer Engineering

Muhammed Yasinhan YAŞAR

